PayFlow Events v2 — Optimized Design & 2-Phase Plan

Nov 7, 2026 · @gautam

v2 keeps the v1 goals (0 lost, 0 phantom, 0 duplicate effects, flat p99 under 40% skew, 0 events stuck behind a poison pill) and adds one OpenTelemetry trace from the Ledger HTTP request to the consumer commit, so every latency number can be split by stage and traced to a cause.

1. Changes from v1

The design is the same. v2 instruments first, splits every latency into stages, and swaps a few defaults for ones Kafka 4 made better.

Area	v1	v2	Why
Observability	Micrometer + JMX exporter → Prometheus	OpenTelemetry (OTLP) → OTel Collector → Prometheus + Tempo + Loki → Grafana	Metrics, traces and logs joined by trace id
Tracing	None	`traceparent` written in the outbox row, carried as a Kafka header by both relays, continued by the consumer	One trace from the Ledger HTTP request to the consumer commit
Latency	One end-to-end number	Split into relay latency (commit → broker append) and consume latency (append → consumer commit) using `message.timestamp.type=LogAppendTime`	Shows whether the relay or the consumer is slow
Lag	Messages behind	Seconds behind, per partition	A 10k-message lag means nothing without the rate

Area	v1	v2	Why
Rebalancing	CooperativeStickyAssignor + static membership	Benchmark 3 options: classic eager, classic cooperative + static, KIP-848 `group.protocol=consumer`	KIP-848 moves assignment to the broker and is GA since Kafka 4.0
Kafka	3.x KRaft	4.1.x (KRaft only)	Current line, supported by Debezium 3.4+
Debezium	Unpinned	3.4+	Adds PostgreSQL 18 and Kafka 4.1.1 support
Java / Boot / Postgres	21 / 3 / 16	25 / 4.0.x / 18	Official OTel starter, no virtual-thread pinning, native `uuidv7()`
Source of events	“The ledger’s outbox table”	Shared outbox contract from PayFlow Ledger. A small `source-app` writes the same schema for standalone runs	Events can be tested without the Ledger, and plugs into it unchanged
Dedup writes	One INSERT per event (efficiency lever)	Batched multi-row `INSERT ... ON CONFLICT DO NOTHING RETURNING` as tuning step T3	Measured, not assumed
Rollups	At-least-once Kafka Streams	`processing.guarantee=exactly_once_v2`	Stage-1 and stage-2 sums never double-count
Results	Manual results.md	Verifier and benches tag every run with `run_id`. `make report` queries Prometheus into results.json + results.md	Every number is reproducible from one command

Area	v1	v2	Why
Serialization	Avro	JSON payload + JSON Schema in the registry, zstd compression	The Ledger outbox payload is jsonb and Debezium passes it through unchanged. One contract, no converter. Bytes per event are measured; Avro stays a stretch goal

2. Stack v2

Pin minor versions in the Maven BOM and docker-compose.yml. Upgrade only between phases.

Layer	Choice	Why
Language	Java 25 LTS, virtual threads for relay and consumer workers	Same as PayFlow Ledger, no pinning on `synchronized`
Framework	Spring Boot 4.0.x, Spring for Apache Kafka 4.x	`@RetryableTopic` non-blocking retries + DLQ, Kafka observation for trace propagation
Broker	Apache Kafka 4.1.x, KRaft, 3 brokers	RF=3, `min.insync.replicas=2`, real broker-kill test
CDC	Debezium 3.4+ Postgres connector on Kafka Connect, Outbox Event Router SMT	Reads the WAL, no polling load
Source DB	PostgreSQL 18, `wal_level=logical`	Outbox + replication slot + `processed_events` dedup store

Layer	Choice	Why
Serialization	JSON payload + JSON Schema in Apicurio Registry 3, zstd on the wire	The Ledger outbox payload is jsonb, which the Debezium router passes through unchanged. Avro would need a bytea Avro column written by the Ledger. Schema evolution is still checked in CI, and bytes per event is measured
Aggregation	Kafka Streams, `exactly_once_v2`, classic rebalance protocol	KIP-1071 Streams protocol is early access in 4.1, so not used
Producer	`enable.idempotence=true`, `acks=all`, zstd	No broker-side duplicates on retry
Consumer	Manual offset commit after DB commit, rebalance protocol under test	Effect is exactly-once
Telemetry SDK	`spring-boot-starter-opentelemetry`, Kafka client metrics via Micrometer, `opentelemetry-logback-appender`	One starter for all three signals
Telemetry backend	OTel Collector (contrib), Prometheus 3, Tempo, Loki, Grafana 12	Provisioned from the repo
Kafka telemetry	Collector `kafkametrics` receiver (offsets, consumer-group lag), Prometheus JMX exporter agent on brokers and Connect, scraped by the Collector	Broker health, Debezium lag, partition load
DB telemetry	Collector `postgresql` + `sqlquery` receivers, `docker_stats`	Slot lag, outbox backlog, DB CPU for polling vs CDC
Tests	JUnit 5, Testcontainers (Kafka, Postgres, Debezium), Awaitility, Toxiproxy	Real broker and DB, fault injection
Load	Java Zipf + spike generator, Gatling for HTTP mode against PayFlow Ledger	Reproducible skew with a fixed seed
Ops UI	Kafka UI (provectus)	Topics, offsets, DLQ contents

Layer	Choice	Why
Build and ship	Maven, Jib, Docker Compose, GitHub Actions	One-command stack, CI gates

3. Architecture

An event exists only if its outbox row committed. A relay copies committed rows to Kafka at least once. Consumers dedupe by event id, so delivery is at-least-once and the effect is exactly-once.

Record shape (both relays must produce it byte-for-byte)

Kafka part	Outbox column	Purpose
key	aggregate_id	Partition key: per-payment order
value	payload (jsonb, as-is)	Event body, validated against the registered JSON Schema
header `id`	id (uuidv7)	Global dedup key
header `type`	type	PaymentAuthorized, PaymentCaptured, TransferPosted, …
header `seq`	seq	Gap and reorder detection per aggregate
header `occurred_at`	occurred_at	Start of end-to-end latency
header `traceparent`	traceparent	Continues the Ledger request trace
header `merchant_id`	merchant_id	Rollups only, never the key
topic	aggregate_type → `<type>.events`	`payment.events`, `transfer.events`

A contract test feeds the same outbox row to both relays and asserts identical key, value and headers.

Topics (created by an init container, `auto.create.topics.enable=false`)

Topic	Key	Partitions	Retention	Config
payment.events, transfer.events	aggregate_id	12 (sweep 6–48)	7 days	`message.timestamp.type=LogAppendTime`
payment.events-retry-5s, -retry-1m	same	12	1 day	Non-blocking retries
payments.dlq	same	3	30 days	After 3 failed attempts. Replay CLI
merchant.rollup.stage1	merchant_id#bucket	12	1 day	Salted partial sums per 1 s window
merchant.totals	merchant_id	6	compacted	Final totals

All topics: `replication.factor=3`, `min.insync.replicas=2`. `@RetryableTopic(autoCreateTopics = "false")`.

Relay A: polling publisher (Phase 1)

1. `SELECT ... FROM outbox WHERE published_at IS NULL ORDER BY id LIMIT 500 FOR UPDATE SKIP LOCKED`.
2. Send the batch with the idempotent producer. Wait for every ack.
3. `UPDATE outbox SET published_at = now()` for the batch, commit.
4. A crash between 2 and 3 re-sends the batch. Consumers dedupe.
5. Idle backoff: 5 ms after a full batch, 50 ms after an empty one. Poll interval is a benchmark variable.

Relay B: Debezium CDC (Phase 2)

```
{
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "plugin.name": "pgoutput",
  "table.include.list": "public.outbox",
  "heartbeat.interval.ms": "5000",
  "value.converter": "org.apache.kafka.connect.json.JsonConverter",
  "value.converter.schemas.enable": "false",
  "transforms": "outbox",
  "transforms.outbox.type": "io.debezium.transforms.outbox.EventRouter",
  "transforms.outbox.table.field.event.id": "id",
  "transforms.outbox.table.field.event.key": "aggregate_id",
  "transforms.outbox.table.field.event.payload": "payload",
  "transforms.outbox.table.expand.json.payload": "true",
  "transforms.outbox.route.by.field": "aggregate_type",
  "transforms.outbox.route.topic.replacement": "${routedByValue}.events",
  "transforms.outbox.table.fields.additional.placement":
"type:header:type,seq:header:seq,occurred_at:header:occurred_at,traceparent:h
eader:traceparent,merchant_id:header:merchant_id"
}
```

In CDC mode the source inserts and deletes the outbox row in the same transaction (`outbox.mode=delete`). The WAL still carries the insert, and the table stays near-empty. Heartbeats keep the slot advancing on a quiet DB. Verify every key against the Debezium 3.4 docs before relying on it.

Consumer: effectively-once

1. Poll a batch, auto-commit off.
2. One Postgres transaction per batch: `INSERT INTO processed_events (event_id) VALUES ... ON CONFLICT DO NOTHING RETURNING event_id`. Apply side effects only for returned ids. Check `seq` against `aggregate_seq` for each aggregate. Commit.
3. Commit Kafka offsets after the DB commit. A crash in between redelivers, which step 2 absorbs.
4. A failing event throws to `@RetryableTopic`: retry-5s, retry-1m, then payments.dlq. The main partition keeps flowing.
5. `processed_events` is partitioned by day. Drop partitions older than topic retention + 1 day.

Skew strategy

Ordering is needed per payment, not per merchant, so the key is `aggregate_id` and a hot merchant spreads across all partitions. Merchant totals use two stages: stage 1 keys by `merchant_id#(hash(payment_id) % B)` and sums per 1 s window; stage 2 re-keys by `merchant_id` and adds the B partials. Both stages run with `exactly_once_v2`.

Trace propagation

The Ledger writes the request span's W3C `traceparent` into the outbox row. Both relays copy it into a Kafka header. The consumer's Spring Kafka observation (`spring.kafka.listener.observation-enabled=true`) reads the header and starts its span as a child. One Tempo trace shows HTTP → Postgres commit → broker append → consumer commit.

4. Observability: OpenTelemetry → Grafana

Apps push OTLP to one Collector. The Collector also pulls Kafka, Connect, Postgres and container metrics, so one Prometheus holds every number in the results.

Pipeline

Signal	Source	Collector	Store
App metrics	Micrometer OTLP registry in relay-poller, consumer, rollup-streams, source-app, verifier, generator	`otlp` → `memory_limiter, resource, batch` → `otlphttp/prometheus`	Prometheus 3 (OTLP receiver, exemplar storage)
Kafka client metrics	Spring Kafka Micrometer producer/consumer listeners (auto-bound)	same	Prometheus
Broker + Connect + Debezium JMX	Prometheus JMX exporter javaagent on each broker and on Connect (port 7071)	`prometheus` receiver scrapes them	Prometheus
Offsets and group lag	`kafkametrics` receiver (brokers, topics, consumers scrapers)	same	Prometheus

Signal	Source	Collector	Store
Postgres	`postgresql` receiver + `sqlquery` (slot lag, outbox backlog, oldest unpublished age, processed_events size)	same	Prometheus
Container CPU and memory	`docker_stats` receiver	same	Prometheus
Traces	Micrometer Tracing OTel bridge, Kafka observation, JDBC spans	`otlp` → `tail_sampling` → `otlp/tempo`	Tempo
Logs	Logback JSON + `OpenTelemetryAppender`	`otlp` → `otlphttp/loki`	Loki

Latency split (the key measurement)

```
T_{e2e} = \underbrace{(t_{append} - t_{occurred})}_{\text{relay}} + \underbrace{(t_{commit} - t_{append})}_{\text{consume}}
```

`t_occurred` is the outbox `occurred_at` header. `t_append` is the record timestamp, set by the broker because topics use `LogAppendTime`. `t_commit` is taken after the consumer's DB commit. All containers share one host clock, so the stages add up without skew. In multi-host deploys, report only the end-to-end number.

App config (consumer example)

```
spring.kafka:
  listener.observation-enabled: true
  template.observation-enabled: true
management:
  otlp.metrics.export: { url: http://otel-collector:4318/v1/metrics, step: 5s
}
  opentelemetry:
    tracing.export.otlp.endpoint: http://otel-collector:4318/v1/traces
    logging.export.otlp.endpoint: http://otel-collector:4318/v1/logs
    resource-attributes:
      service.name: payflow-consumer
      run_id: ${RUN_ID:adhoc}
      scenario: ${SCENARIO:none}
      relay: ${RELAY:poller}
  tracing.sampling.probability: 1.0
  metrics.distribution.slo:
    payflow.events.e2e.latency: 10ms,25ms,50ms,100ms,200ms,500ms,1s,2s,5s
    payflow.events.relay.latency: 5ms,10ms,25ms,50ms,100ms,200ms,500ms,1s
    payflow.events.consume.latency: 5ms,10ms,25ms,50ms,100ms,200ms,500ms,1s
```

Rules

- **Naming.** OTel dots (`payflow.events.e2e.latency`), Prometheus underscores (`payflow_events_e2e_latency_seconds`). Queries here use the Prometheus form.
- **Cardinality.** `partition` is allowed (at most 48). Never label by `event_id`, `payment_id`, `aggregate_id` or `merchant_id`. Hot-merchant share is computed by the generator and reported per run.
- **Run tagging.** Promote `service.name`, `run_id`, `scenario` and `relay` to labels. Every script sets `RUN_ID`, `SCENARIO`, `RELAY` first.
- **Lag in time.** Alert on seconds behind, not messages behind.
- **Tail sampling.** Keep 100% of ERROR traces and traces longer than 200 ms, plus 2% of the rest. 100% during chaos and poison-pill runs.
- **Overhead budget.** Telemetry on vs off at 5k events/s: p99 difference under 5%.
- **As code.** Collector, Prometheus rules, Grafana dashboards and alert rules live in `deploy/observability/`.

5. Metrics catalog

Custom metrics live in one `EventsMetrics` class in `common/`, so every service uses the same names.

Latency and flow (custom)

Metric (Prometheus name)	Type	Labels	Used for
`payflow_events_e2e_latency_seconds`	histogram	topic, relay, poison=false|true	Headline p99, SLO, healthy-event p99 during pills
`payflow_events_relay_latency_seconds`	histogram	topic, relay	Commit → broker append
`payflow_events_consume_latency_seconds`	histogram	topic	Append → consumer commit
`payflow_events_consumer_lag_seconds`	gauge	topic, partition	Now minus append time of the record being processed
`payflow_events_processed_total`	counter	topic, result=applied|duplicate|failed	Throughput, duplicates received
`payflow_events_order_violations_total`	counter	kind=gap|reversal	Must stay 0
`payflow_events_dedup_batch_size`	distribution summary	—	Tuning step T3
`payflow_events_dedup_insert_seconds`	histogram	—	Dedup store cost
`payflow_events_retry_total`	counter	stage=retry-5s|retry-1m|dlq	Poison-pill path
`payflow_events_dlq_replayed_total`	counter	result	Replay CLI
`payflow_events_consumption_gap_seconds`	histogram	—	Longest pause between processed events (rebalance pause)

Relays

Metric	Source	Used for
`payflow_relay_batch_size`, `payflow_relay_poll_seconds`, `payflow_relay_published_total`	relay-poller (custom)	Poller efficiency
`pg_outbox_backlog_rows`, `pg_outbox_oldest_unpublished_seconds`	sqlquery receiver	Poller falling behind
`debezium_metrics_millisecondsbehindsource`	JMX exporter on Connect	CDC lag
`debezium_metrics_queueremainingcapacity`, `_totalnumberofeventsseen`, `_millisecondssincelastevent`	JMX exporter on Connect	CDC queue pressure, liveness
`pg_replication_slot_lag_bytes{slot}`	sqlquery: `pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)`	WAL retained for the slot
`container_cpu_utilization{container="postgres"}`	docker_stats	DB CPU, polling vs CDC

Kafka clients (Micrometer, built-in names)

Metric	Used for
`kafka_producer_record_send_rate`, `_record_error_rate`, `_record_retry_rate`	Producer health
`kafka_producer_batch_size_avg`, `_compression_rate_avg`, `_request_latency_avg`, `_record_queue_time_avg`	linger/batch/zstd tuning
`kafka_consumer_fetch_manager_records_lag_max`, `_records_consumed_rate`, `_fetch_latency_avg`	Consumer health
`kafka_consumer_coordinator_rebalance_total`, `_rebalance_latency_avg`, `_last_rebalance_seconds_ago`	Rebalance benchmark

Brokers and offsets

Metric	Source	Used for
`kafka_consumer_group_lag{group,topic,partition}`	kafkametrics receiver	Lag in messages, per partition
`kafka_partition_current_offset{topic,partition}`	kafkametrics receiver	Per-partition produce rate, imbalance
`kafka_server_replicamanager_underreplicatedpartitions`	broker JMX	Broker-kill test
`kafka_controller_kafkacontroller_offlinepartitionscount`	broker JMX	Must be 0 outside the kill window
`kafka_server_replicamanager_isrshrinks_total`	broker JMX	ISR churn
`kafka_server_brokertopicmetrics_bytesin_total`, `_bytesout_total`	broker JMX	Bytes per event with zstd
`kafka_server_kafkarequesthandlerpool_requesthandleravgidlepercent`	broker JMX	Broker saturation

Skew lab and rollups

Metric	Source	Used for
`payflow_generator_sent_total{strategy}`	generator	Offered load
`payflow_generator_hot_share`	generator	Confirms the 40% hot merchant
`kafka_stream_processor_node_process_rate`, `kafka_stream_thread_commit_latency_avg`	Kafka Streams (Micrometer)	Rollup throughput
`payflow_rollup_mismatch_minor`	verifier	Sum of merchant.totals minus source sum, must be 0

Correctness (verifier, set once per run)

Metric	Labels	Pass rule
`payflow_verifier_lost_events`	run_id, scenario, relay	0 for outbox. Counted for dual-write
`payflow_verifier_phantom_events`	same	0 for outbox. Counted for dual-write
`payflow_verifier_duplicate_effects`	same	0
`payflow_verifier_duplicates_received`	same	Reported
`payflow_verifier_order_violations`	same	0
`payflow_verifier_dlq_count`, `payflow_verifier_pills_injected`	same	Equal
`payflow_verifier_last_run_timestamp_seconds`	same	Fresh after every run

6. Dashboards, SLOs and alerts

Seven provisioned dashboards, three SLOs and twelve alert rules. Every dashboard has `run_id`, `scenario` and `relay` variables and shows generator and chaos events as annotations. JMX metric names depend on the exporter rules in `deploy/observability/jmx/*.yml`; those rules must produce the names in section 5.

Dashboards

Dashboard	Top row	Panels below
Pipeline Overview	Events/s in vs out, e2e p99 by relay, max lag (s), DLQ received, verifier status	Processed by result, duplicates received, order violations
Latency Breakdown	Relay p99 vs consume p99	Heatmaps of both, exemplars to Tempo, e2e p50/p99 over time
Relays	Poller backlog and oldest age, Debezium ms behind	Batch size, poll time, Debezium queue capacity, slot lag bytes, Postgres CPU polling vs CDC
Partitions & Skew	Imbalance (max/mean), hot share	Per-partition produce rate heatmap, lag per partition, consumers in group
Consumers & Rebalance	Rebalances, longest consumption gap	Dedup batch size and insert p99, retry and DLQ flow, rebalance latency by protocol
Brokers	Under-replicated, offline partitions	ISR shrinks, bytes in/out, request handler idle %, producer batch and compression ratio
Correctness	Lost, phantom, duplicate effects, order violations per run	Dual-write vs outbox side by side, rollup mismatch, DLQ count vs pills injected

Recording rules (`prometheus/rules/events.yml`)

```
- record: events:e2e_p99:1m
  expr: histogram_quantile(0.99, sum by (le, relay, run_id)
(rate(payflow_events_e2e_latency_seconds_bucket{poison="false"}[1m])))
- record: events:relay_p99:1m
  expr: histogram_quantile(0.99, sum by (le, relay, run_id)
(rate(payflow_events_relay_latency_seconds_bucket[1m])))
- record: events:consume_p99:1m
  expr: histogram_quantile(0.99, sum by (le, run_id)
(rate(payflow_events_consume_latency_seconds_bucket[1m])))
- record: events:throughput:1m
  expr: sum by (relay, run_id)
(rate(payflow_events_processed_total{result="applied"}[1m]))
- record: events:partition_rate:1m
  expr: deriv(kafka_partition_current_offset{topic="payment.events"}[1m])
- record: events:partition_imbalance:1m
  expr: max(events:partition_rate:1m) / avg(events:partition_rate:1m)
- record: events:max_lag_seconds
  expr: max by (run_id) (payflow_events_consumer_lag_seconds)
- record: events:sli_latency_200ms:5m
  expr:
sum(rate(payflow_events_e2e_latency_seconds_bucket{le="0.2",poison="false"}
[5m])) / sum(rate(payflow_events_e2e_latency_seconds_count{poison="false"}
[5m]))
```

Add the latency SLI over `[30m]` , `[1h]` and `[6h]` for burn-rate alerts.

SLOs

SLO	Objective	How
Completeness	0 lost, 0 duplicate effects	Verifier after every run, hard gate
End-to-end latency	99% of healthy events under 200 ms (CDC)	`events:sli_latency_200ms`
Freshness	Max lag under 30 s for 99.9% of minutes	`events:max_lag_seconds`

Alert rules (`grafana/alerting/events-alerts.yml`)

Alert	Expression	For	Severity
LostEvents	`max(payflow_verifier_lost_events{relay!="dual-write"}) > 0`	0m	critical
DuplicateEffects	`max(payflow_verifier_duplicate_effects) > 0`	0m	critical
OrderViolations	`increase(payflow_events_order_violations_total[5m]) > 0`	0m	critical
VerifierStale	`time() - max(payflow_verifier_last_run_timestamp_seconds) > 86400`	10m	warning
ConsumerLagHigh	`events:max_lag_seconds > 30`	2m	page
LatencyFastBurn	`(1 - events:sli_latency_200ms:1h) > 14.4 * 0.01 and (1 - events:sli_latency_200ms:5m) > 14.4 * 0.01`	2m	page
DLQReceiving	`sum(increase(payflow_events_retry_total{stage="dlq"}[5m])) > 0`	5m	warning
DLQNotReplayed	`sum(kafka_consumer_group_lag{group="dlq-replay"}) > 0`	30m	ticket
DebeziumBehind	`max(debezium_metrics_millisecondsbehindsource) > 10000`	2m	page
ReplicationSlotLag	`max(pg_replication_slot_lag_bytes) > 1e9`	5m	page
OutboxBacklogOld	`max(pg_outbox_oldest_unpublished_seconds) > 30`	2m	page
UnderReplicated	`sum(kafka_server_replicamanager_underreplicatedpartitions{scenario!~"chaos.*"}) > 0`	1m	page

7. Phase 1: correct pipeline + broken baseline (weeks 1–2)

Phase 1 ends with the dual-write baseline losing events, the polling outbox losing none, and both measured through the telemetry Phase 2 will use. Start after PayFlow Ledger has its outbox migration (Ledger runbook P1-04).

Day	Deliverable	Done when
1	Multi-module Maven repo, compose: 3 Kafka 4.1 brokers (KRaft), Apicurio, Postgres 18, Kafka UI, topic-init container. Makefile (`up down test it chaos skew pills bench report`)	`make up` healthy. Topics exist with the configs in section 3
2	Telemetry stack: Collector (otlp, kafkametrics, prometheus scrape of JMX agents, postgresql, sqlquery, docker_stats), Prometheus 3, Tempo, Loki, Grafana provisioned	Broker JMX, group lag and Postgres metrics visible in Grafana
3	`common/`: `EventsMetrics`, record shape, header names, JSON Schema registered with BACKWARD compatibility. Outbox migration identical to the Ledger contract, plus `processed_events` (partitioned by day) and `aggregate_seq`	CI fails on an incompatible schema change
4	`source-app` (business row + outbox row in one transaction, fixed rate, `outbox.mode=retain|delete`). `dual-write-app` with two modes: `db-first` (commit then send) and `kafka-first` (send then commit, 0.1% forced rollbacks)	Both write at 5k events/s on the laptop
5	`relay-poller` (SKIP LOCKED, 500-row batches, idempotent producer, metrics). Contract test: record shape per section 3	Contract test green. `pg_outbox_backlog_rows` near 0 at 5k/s
6	`consumer`: per-event dedup insert (baseline), seq check, offsets after DB commit, latency split, trace continued from `traceparent`	One Tempo trace spans source-app → relay → consumer
7	`verifier`: compares committed source event ids with `processed_events` and Kafka; sets the correctness gauges and writes results.json per `run_id`	Seeded test cases for lost, phantom, duplicate, reorder are each detected
8	Crash test v1: `kill -9` the writer every 10–30 s during a 1M-event run. Three runs: dual-write db-first, dual-write kafka-first, outbox + poller	Verifier output for all three runs in results.json

Day	Deliverable	Done when
9	Dashboards: Pipeline Overview, Latency Breakdown, Relays, Correctness. Critical alerts. `make report`	Report regenerates results.md from Prometheus
10	Baseline: 1M uniform events at 1k and 5k events/s, telemetry on and off. CI on push	Baseline table in results.md with machine spec

Phase 1 exit gate

- [ ] Dual-write db-first loses more than 0 events; dual-write kafka-first produces more than 0 phantoms (both counted and published)
- [ ] Outbox + poller: 0 lost, 0 phantom, 0 duplicate effects, 0 order violations across the crash run
- [ ] Baseline throughput and e2e p99, split into relay and consume, in results.md
- [ ] End-to-end trace visible from the Ledger (or source-app) request to the consumer commit
- [ ] Telemetry overhead under 5% of p99
- [ ] CI green: unit, schema compatibility, contract, Testcontainers integration, idempotency, ordering

8. Phase 2: CDC, chaos, skew, poison pills (weeks 3–4)

Phase 2 swaps in CDC, attacks the pipeline, then tunes one change at a time. A change stays only if every correctness metric stays 0 and the measured gain is at least 5%.

Day	Deliverable	Done when
11	Kafka Connect + Debezium 3.4+, connector JSON registered by an init script, `outbox.mode=delete`, JMX agent on Connect	Contract test passes for the CDC relay. Outbox table stays near-empty
12	Polling vs CDC at 5k events/s and at max sustained rate	Table: e2e p99 (relay + consume), events/s, Postgres CPU, slot lag
13–14	Chaos matrix, 1M events per run, verified after each (table below)	`make chaos` runs unattended and prints pass/fail per run

Day	Deliverable	Done when
15–16	Skew lab: 3 key strategies × partitions {6, 12, 24, 48} × consumers {1, P/4, P/2, P} at 40% hot merchant. 10× spike for 60 s. Rollups with `exactly_once_v2`	Curves of events/s, p99, max lag and imbalance. Spike max lag and recovery time. Rollup mismatch = 0
17	Poison pills: 0.1% failing events, retry-5s → retry-1m → DLQ, replay CLI	Healthy-event p99 with pills within 10% of without. DLQ count = injected
18	Rebalance benchmark: classic eager, classic cooperative + static membership, KIP-848 `group.protocol=consumer`. Rolling restart of 4 consumers	Longest consumption gap and total pause per protocol
19	Tuning loop T1–T6 below	Before/after row per step in results.md
20	Nightly CI, README with all charts	Definition of done ticked

Chaos matrix

Scenario	Relays run	Pass rule
`kill -9` the writer every 10–30 s	dual-write (both modes), poller, CDC	Outbox: 0 lost, 0 phantom. Dual-write: counted
`kill -9` 1 of 3 brokers mid-stream, restart after 60 s	poller, CDC	0 lost, 0 duplicate effects, offline partitions = 0
Restart Postgres	poller, CDC	0 lost. Slot resumes, no snapshot re-read
Restart Kafka Connect	CDC	0 lost. Duplicates received are counted, duplicate effects = 0
`kill -9` a consumer mid-batch	poller, CDC	0 duplicate effects, 0 order violations
Toxiproxy latency 100 ms + reset_peer between consumer and Postgres	poller, CDC	0 duplicate effects. Lag recovers after the toxic is removed

Run hygiene

- Reset state per run: truncate `processed_events`, recreate topics from the init script, reset the connector offsets.
- Fixed generator seed and rate. 60 s warm-up excluded from the measure window.
- One `run_id` per run; `make report` builds every table and chart from Prometheus and results.json.
- Sweeps resume from the last completed run if interrupted.

Tuning loop (one change per step, measured at 5k events/s uniform and at 40% skew)

Step	Change	Watch
T1	Producer `linger.ms` {5, 10, 20} × `batch.size` {64, 128, 256 KB}, zstd	`kafka_producer_batch_size_avg`, `_compression_rate_avg`, relay p99
T2	Poller batch {100, 500, 2000} and idle backoff	`payflow_relay_poll_seconds`, Postgres CPU
T3	Batched dedup: one multi-row INSERT ... RETURNING per poll	`payflow_events_dedup_insert_seconds`, consume p99
T4	`max.poll.records` {100, 500, 1000}, `fetch.min.bytes` 64 KB with `fetch.max.wait.ms` 10	Consume p99, events/s
T5	Listener concurrency up to partition count per instance	Events/s per CPU
T6	Confluent Parallel Consumer, key-ordered (optional)	Events/s beyond the partition ceiling, 0 order violations

Phase 2 exit gate

- [ ] Every chaos run: outbox relays at 0 lost, 0 phantom, 0 duplicate effects, 0 order violations
- [ ] Polling vs CDC table and chart published
- [ ] Skew curves, spike recovery and rebalance results published
- [ ] Tuning table complete, kept and reverted steps recorded
- [ ] Nightly CI green three nights in a row

9. Target metrics

Correctness rows must be exactly 0. Performance rows are aims: publish what you measure, with the machine spec.

Metric	Target	Measured by	Gate
Lost events, outbox	0 in every chaos run	`payflow_verifier_lost_events{relay=~"poller|cdc"}`	CI hard fail
Lost events, dual-write	Counted and published	`payflow_verifier_lost_events{relay="dual-write"}`	Headline contrast
Phantom events	0 for outbox, counted for dual-write	`payflow_verifier_phantom_events`	CI hard fail (outbox)
Duplicate side effects	0	`payflow_verifier_duplicate_effects`	CI hard fail
Ordering violations	0	`payflow_verifier_order_violations` + `payflow_events_order_violations_total`	CI hard fail
Rollup mismatch	0 minor units	`payflow_rollup_mismatch_minor`	CI hard fail
DLQ count vs injected	Equal	`payflow_verifier_dlq_count` = `payflow_verifier_pills_injected`	CI hard fail
e2e p99, CDC, 5k events/s	Under 200 ms	`events:e2e_p99:1m{relay="cdc"}`	Report
e2e p99, polling vs CDC	Report both, split into relay + consume	`events:relay_p99:1m`, `events:consume_p99:1m`	Report
Throughput, 12 partitions	10k events/s or more after tuning	`events:throughput:1m`	Report
Partition imbalance, 40% skew	Under 1.5 with entity key. merchant_id key reported	`events:partition_imbalance:1m`	Report
p99 under skew	Within 2× of uniform at the same rate	`events:e2e_p99:1m` by scenario	Report
Spike recovery (10× for 60 s)	Under 2 min back to baseline lag	`events:max_lag_seconds` peak and time to < 1 s	Report
Healthy p99 with poison pills	Within 10% of no-pill run; 0 healthy events delayed past 1 s by a pill	`events:e2e_p99:1m` with `poison="false"`	Report

Metric	Target	Measured by	Gate
Rebalance pause	Reduction vs eager, per protocol	max `payflow_events_consumption_gap_seconds`	Report
Postgres CPU, polling vs CDC	Report	`container_cpu_utilization{container="postgres"}`	Report
Bytes per event	Report, with zstd	`rate(kafka_server_brokertopicmetrics_bytesin_total[1m]) / events:throughput:1m`	Report
Telemetry overhead	Under 5% of p99	Telemetry on vs off	Report

10. Definition of done, repo layout, sources

Definition of done

- [ ] `make up` starts brokers, registry, Postgres, Connect, apps and the telemetry stack on a clean machine
- [ ] Grafana opens with seven provisioned dashboards, no manual setup
- [ ] `make chaos` prints lost, phantom and duplicate counts for dual-write vs outbox in every scenario
- [ ] `make skew` and `make pills` write their curves; `make report` regenerates results.md from Prometheus
- [ ] README: architecture diagram, results table, polling-vs-CDC chart, skew chart, rebalance chart, tuning table, machine spec
- [ ] DLQ replay CLI documented with an example
- [ ] One trace from PayFlow Ledger `POST /v1/transfers` to the consumer commit, screenshot in README
- [ ] CI green, nightly chaos report attached

Repo layout

```
payflow-events/
  CLAUDE.md
  Makefile
  common/          record shape, header names, JSON Schema, EventsMetrics,
salting
  source-app/      business row + outbox in one tx (standalone mode)
  dual-write-app/  db-first and kafka-first baselines, experiment only
  relay-poller/    SKIP LOCKED relay
  consumer/        idempotent consumer, @RetryableTopic, DLQ replay CLI
  rollup-streams/  two-stage merchant totals, exactly_once_v2
  verifier/        lost / phantom / duplicate / order / rollup oracle
  generator/       Zipf + spike generator
  connect/         debezium-outbox.json, register.sh
  chaos/           kill, broker-stop, connect-restart, toxiproxy scripts
  perf/            skew matrix, report.py
  deploy/docker-compose.yml
  deploy/observability/  otel-collector.yaml, jmx/, prometheus/, tempo/,
loki/, grafana/
  deploy/helm/, deploy/strimzi/
  docs/            architecture.png, results.md, results.json
```

Sources

- OpenTelemetry with Spring Boot (Spring team): starter, OTLP export properties, log appender setup
- Apache Kafka 4.1.0 release announcement: Queues for Kafka in preview, Streams rebalance protocol KIP-1071 early access, built on KIP-848
- Debezium 3.4.0.Final summary: Kafka 4.1.1 and PostgreSQL 18 support
- PostgreSQL 18 released: `uuidv7()`, async I/O